Advanced Terraform on Azure
Lab 2 – Hygiene and Validation

Expected Duration

This lab should take 60-70 minutes to complete

Teaching Points

By the end of this lab, you will be able to:

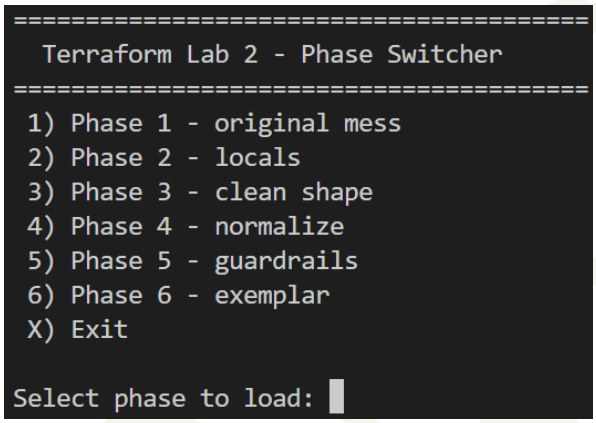
- Recognize the risks of poor hygiene and validation in Terraform configurations.
- Understand how inconsistent keys can cause configuration churn.
- Use locals to enforce naming conventions and tag consistency.
- Apply string and collection functions to normalize inputs.
- Implement guardrails using variable validation and lifecycle conditions.
- Use terraform console to safely inspect and test expressions.
- Produce deterministic, predictable Terraform configurations.

Before You Begin

The lab2 folder used for this lab contains three sub-directories; '**guided**', '**original**' and '**phases**'.

The **guided** directory is where you'll complete the structured walkthrough. The **original** directory holds the untouched starter files for comparison if needed. To reset this lab if your encounter issues; first destroy any resources created during this lab, then delete the entire **guided** folder content before copying the **original** folder content into **guided**.

When working in the **guided** directory, you have access to a script named '**phases.cmd'**. Running this 'Phase switcher' script allows you to switch between lab phases, as outlined in the instructions which follow. Selecting a phase will copy new files into the guided folder, simulating progressive modifications to the original 'messy' files. The providers.tf file which exists in the guided folder initially will remain static throughout the lab…



Important: Do not have any **guided** files active in VSC editor as you switch between phases. Doing so can cause 'On-disk vs In-memory' file version conflicts and may lead to 'phantom' error displays. If any of these appear then simply close and re-open VSC to clear them. All the supplied code has been extensively tested and works 'as-is'

You can use the 'compare' feature in VSC to view the file differences as you move through this lab.

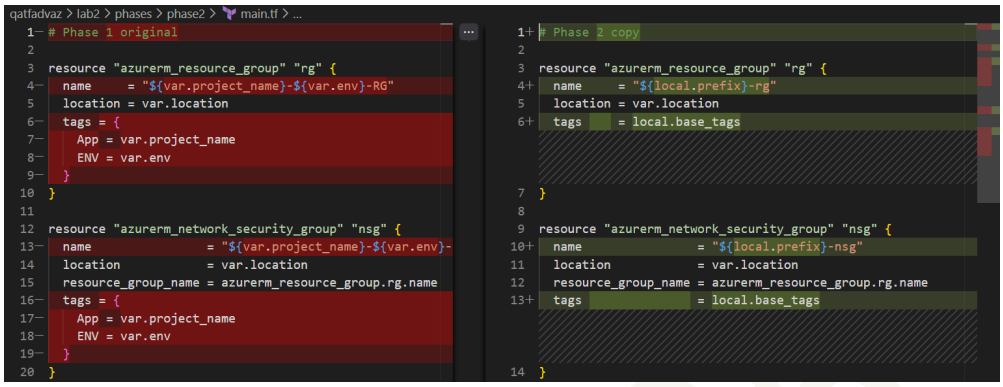
Try this now; to compare main.tf changes from Phase 1 to Phase 2 ..

In explorer, navigate to Phases/Phase1 and right click main.tf

Select "Select for compare"

Navigate to Phases/Phase2 and right-click main.tf

Select “Compare with selected”



Lab rules

Do not make any changes to terraform.tfvars. In real projects, tfvars files are often owned or populated by another team or by a pipeline. Your Terraform code must therefore sanitize provided input, normalize it and enforce guardrails without assuming the input is perfect or that you can change it.

The code provided deploys 4 resources; a resource groups and an NSG with 2 rules. The rules assigned to the NSG have source_address_prefixes, either directly assigned, drawn from global allowlists, or both. Two sample allowlists exist for http and https traffic. Corporate Network Security mandate that wildcard destination ports are not permitted in NSG rules and that there should be no source port filtering.

Phase 1 – The Untidy Plan

1. Move into the **lab2\guided** subfolder and update **providers.tf** with your subscription_id.
2. Run **phases.cmd** and select Phase 1 ‘original mess’.
3. Run **az login** if necessary.
4. Review the provided files to gain an understanding of the original code and the intended resource provisioning. This understanding is important in order to appreciate the changes you will make.
5. Focus on the ‘messy’ terraform.tfvars …
    - The http/https allowlists contain duplicates and whitespace

- NSG rule keys are in mixed styles (allow_http_from_office, Allow-HTTPS-From-Office)
- NSG rule source_cidrs contains duplicates and whitespace
- NSG rule allow_group uses different casing (HTTP vs https).

6. Attempt to apply this configuration …

```
terraform init
terraform apply --auto-approve
```

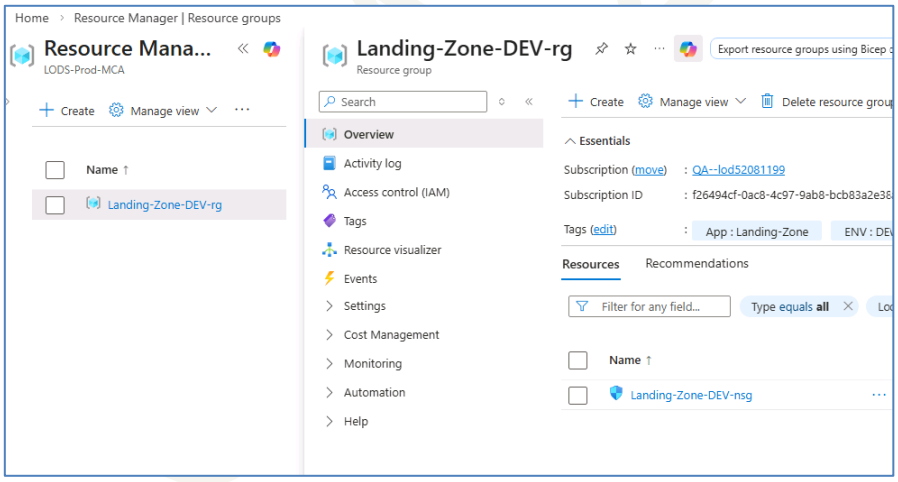
7. Note the errors …

```
azurerm_network_security_rule.rule["Allow-HTTPS-From-Office"]: Creating...

  Error: creating Security Rule (Subscription: "f26494cf-0ac8-4c97-9ab8-bcb83a2e38a9"
  Resource Group Name: "Landing-Zone-DEV-rg"
  Network Security Group Name: "Landing-Zone-DEV-nsg"
  Security Rule Name: "allow_http_from_office"): performing CreateOrUpdate: unexpected
-0ac8-4c97-9ab8-bcb83a2e38a9/resourceGroups/Landing-Zone-DEV-rg/providers/Microsoft.Net
 Value provided:  10.0.10.0/24

    with azurerm_network_security_rule.rule["allow_http_from_office"],
    on main.tf line 22, in resource "azurerm_network_security_rule" "rule":
    22: resource "azurerm_network_security_rule" "rule" {
```

8. Switch to the Azure Portal and verify that some of the configured resources have been deployed …



9. Partial deployment is a strong anti-pattern when using terraform. Destroy the deployed resources …

```
terraform destroy --auto-approve
```

10. Whilst there were errors when we attempted to apply the configuration, these issues may not be captured during a planning phase. Terraform may plan successfully without clean input….

terraform plan -out=out1
terraform show -json out1 | jq > out1.txt

The -out switch generates a binary output of the proposed state configuration. This binary file is then converted to a txt file for later use.

Terraform does not validate input or enforce consistency by default. This 'messy' baseline code highlights the need for strong configuration control.

Phase 2 – Introducing Locals

You will now begin restructuring the configuration to centralize common variables. This will be a multi-phase approach. The goal is to ...

- Centralize the generation of the name given to the resources.
- Centralize the base tags being applied to the resources.

11. Close any open files.
12. Run **phases.cmd** and select 'Phase 2 - locals'
13. A new **locals.tf** file is added and **main.tf** is updated.
14. Review **guided/locals.tf** ...

```
locals {
  env_canon = join("-",regexall("[a-z0-9]+",lower(trimspace(var.env))))
  project_canon = join("-",regexall("[a-z0-9]+",lower(trimspace(var.project_name))))
  prefix        = "${local.project_canon}-${local.env_canon}"

  base_tags = {
    app = local.project_canon
    env = local.env_canon
  }
}
```

The new locals.tf file generates sanitized local variables; it is also used to take messy input in tfvars files and turn it into a version that can be used throughout;

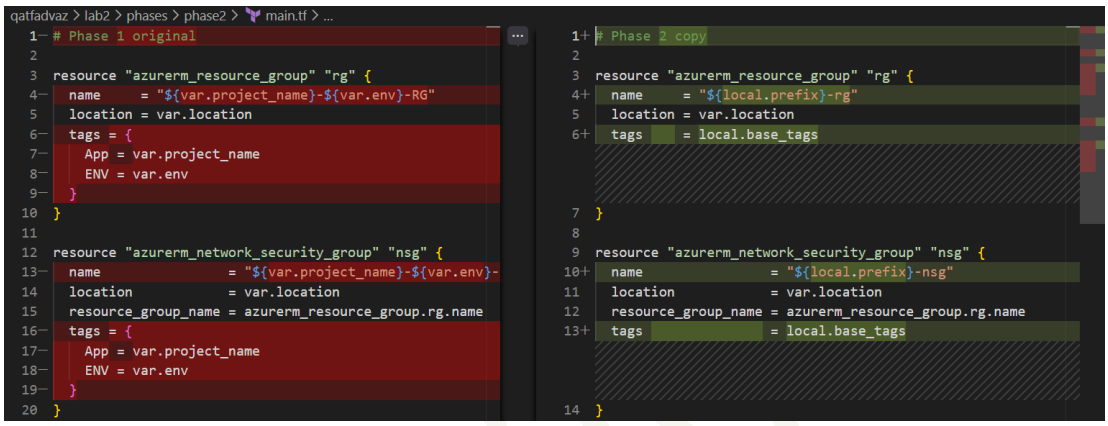
env_canon = join("-",regexall("[a-z0-9]+",lower(trimspace(var.env))))
This expression converts the environment name into a safe, canonical identifier. It normalizes case, removes whitespace and unsupported characters, and produces a lowercase, hyphen-separated value suitable for use in resource names and tags.

project_canon = join("-",regexall("[a-z0-9]+",lower(trimspace(var.project_name))))
The project name is normalized into a predictable, deployment-safe format to ensure consistent resource naming. Normalising the project name ensures that

cosmetic differences in user input do not result in different resource names or unexpected plan changes.

prefix = "${local.project_canon}-${local.env_canon}"
A standard prefix is created from the canonical project and environment names and reused for all resources.

15. Compare the current **main.tf** against the previous **phase1\main.tf** ...



In Phase1main.tf, the original resource names were a repetitive joining of 2 raw variables with a suffix. Now, in Phase 2, the new names are a single local variable, derived from sanitization of these raw variables, with a suffix.

The original tags are another repetitive reference to raw variables. The amount of repetitive, duplicated code is reduced by using a single local variable derived from sanitization of these raw inputs

16. Run planning ...

    terraform plan -out=out2
    terraform show -json out2 | jq > out2.txt

17. Compare **out2.txt** against **out1.txt** and verify that naming and tags are now consistent across all resources...

Locals provide a central reference for clean, consistent naming and tagging — the first step toward eliminating configuration churn.

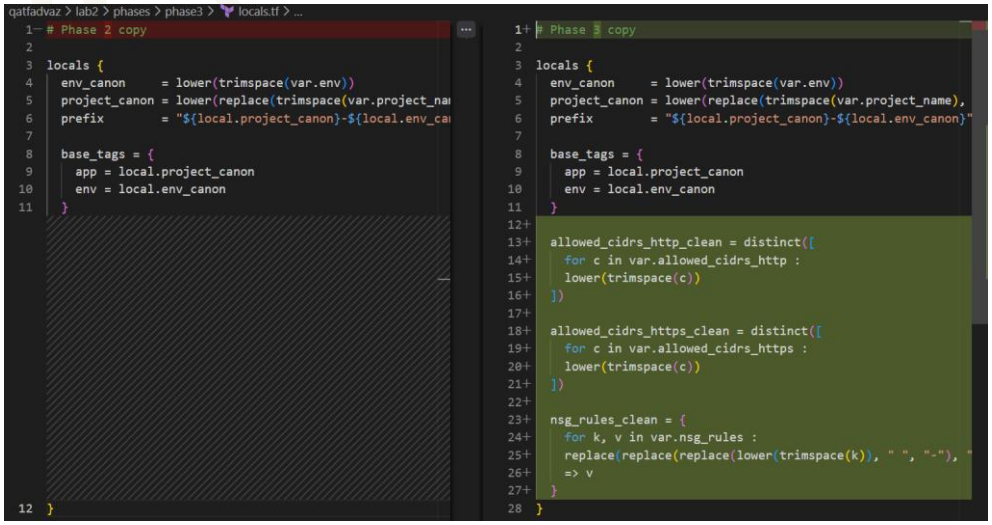
Phase 3 – Cleaning the Data Shape

Resource naming and tagging need to be centralized for consistency. The NSG rule keys and allowed_CIDRs need cleaning and normalizing to prevent churn.

Goals

- Clean http/https lists
- Clean rule keys

18. Close any open files.
19. Run **phases.cmd** and select 'Phase 3 - clean shape'
20. The files **locals.tf** and **main.tf** are updated.
21. Review the current **locals.tf** against **phase2\locals.tf**. Three new local variables that are introduced …



```
allowed_cidrs_http_clean = distinct([
  for c in var.allowed_cidrs_http :
  lower(trimspace(c))
])
```

This cleans and normalizes the list of allowed HTTP CIDR ranges. Each entry is trimmed and converted to lowercase, and duplicate values are removed to ensure the final list is consistent and predictable.

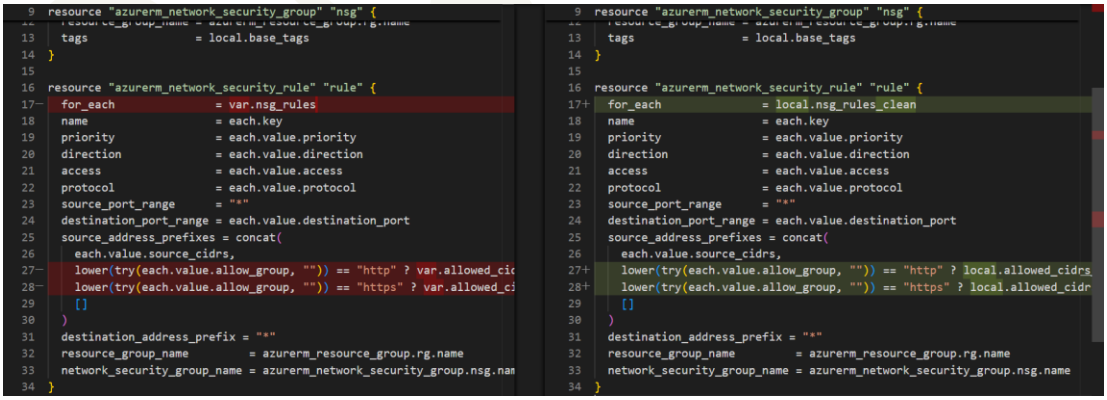
```
allowed_cidrs_https_clean = distinct([
  for c in var.allowed_cidrs_https :
  lower(trimspace(c))
])
```

This cleans and normalizes the list of allowed HTTPS CIDR ranges. Each entry is trimmed and converted to lowercase, and duplicate values are removed to ensure the final list is consistent and predictable.

```
nsg_rules_clean = {
  for k, v in var.nsg_rules :
  replace(replace(replace(lower(trimspace(k)), " ", "-"), "_", "-"), ".", "-")
  => v
}
```

This normalizes NSG rule names into a safe, consistent format. Rule names are trimmed, converted to lowercase, and any spaces or special separators are replaced with hyphens to ensure they are suitable for use as stable resource identifiers.

22. Compare the current **main.tf** against **phase2\main.tf** ...



Previously, main.tf used for_each based on raw input values (var.nsg_rules) to create rules and define globally allowed source addresses (var.allowed_cidrs_http and var.allowed_cidrs_https)

Now, main.tf uses rules and addresses based on cleaned raw inputs from newly defined local variables.

23. Use **terraform console** to ensure keys are slugged, and that CIDRs are cleaned and deduplicated

    keys(local.nsg_rules_clean)
    local.allowed_cidrs_http_clean
    local.allowed_cidrs_https_clean

```
PS C:\qatfadvaz\lab2\guided> terraform console
> keys(local.nsg_rules_clean)
[
  "allow-http-from-office",
  "allow-https-from-office",
]
> local.allowed_cidrs_http_clean
tolist([
  "10.0.0.0/24",
  "10.0.1.0/24",
  "10.0.2.0/24",
])
> local.allowed_cidrs_https_clean
tolist([
  "10.0.1.0/24",
  "10.0.3.0/24",
])
> ^C
```

24. Run planning …

    terraform plan -out=out3
    terraform show -json out3 | jq > out3.txt

25. Compare **out3.txt** against **out2.txt** and verify that naming, tags are consistent across all resources and data has been scrubbed …

Cleaning keys and CIDRs eliminates resource churn and enforces deterministic input structures.

Phase 4 – Normalizing Values

After cleaning keys, we now want to ensure the values themselves are standardized.

Values inside the rules (direction, access, protocol, allow_group) may also be messy in tfvars (“INBOUND”, “ALLOW”, “TCP”, “HTTP” etc..) We want to normalize those too.

26. Close any open files.
27. Run **phases.cmd** and select ‘Phase 4 - normalize’
28. Review the new local variable **nsg_rules_normalised** introduced into **guided\locals.tf**. Phase 3 created a cleaned map of the rules; *nsg_rules_clean*, ensuring the rule *names* are slugged and valid but leaving the object *values* as-is. We now take this clean map and use ***for_in*** to sanitize the object values themselves, thus creating a normalized map. This new normalized map is then used in main.tf

```
29    nsg_rules_normalised = {
30      for name, rule in local.nsg_rules_clean :
31      name => {
32        priority    = rule.priority
33        direction   = lower(rule.direction)
34        access      = lower(rule.access)
35        protocol    = lower(rule.protocol)
36        allow_group = lower(try(rule.allow_group, ""))
37
38  >     source_cidrs = distinct([ ···
42        ])
43
44  >     destination_ports = sort(distinct(compact([ ···
60        ])))
61
62  >     destination_ports_dropped = [ ···
72        ]
73
74  >     source_cidrs_dropped = [ ···
80        ]
81      }
82    }
```

This block normalises the core attributes of each NSG rule. Direction, access, and protocol values are converted to lowercase to ensure consistent behaviour, and optional fields are handled safely using try() so missing values do not cause evaluation errors.

```
38          source_cidrs = distinct([
39            for c in rule.source_cidrs :
40            lower(trimspace(c))
41            if can(cidrnetmask(trimspace(c)))
42          ])
```

This expression cleans and validates source CIDR entries for each rule. Each value is trimmed and normalized to lowercase, and only entries that can be successfully interpreted as valid CIDR ranges are retained.

```
destination_ports = sort(distinct(compact([
  for p in try(rule.destination_ports, []) : (
    length(regexall("[^0-9 -]", trim(p, " -"))) == 0 ? (
      length(regexall("[0-9]+", trim(p, " -"))) == 1 ?
        regexall("[0-9]+", trim(p, " -"))[0] :
      length(regexall("[0-9]+", trim(p, " -"))) == 2 ?
        "${min(
          tonumber(regexall("[0-9]+", trim(p, " -"))[0]),
          tonumber(regexall("[0-9]+", trim(p, " -"))[1])
        )}-${max(
          tonumber(regexall("[0-9]+", trim(p, " -"))[0]),
          tonumber(regexall("[0-9]+", trim(p, " -"))[1])
        )}" :
      ""
    ) : ""
  )
])))
```

Here, destination port input is cleaned and normalized before use. Each entry is checked for valid characters, converted into either a single port or a port range, and automatically ordered where a range is provided. Invalid or ambiguous entries are ignored, and the final list is made unique and deterministic.

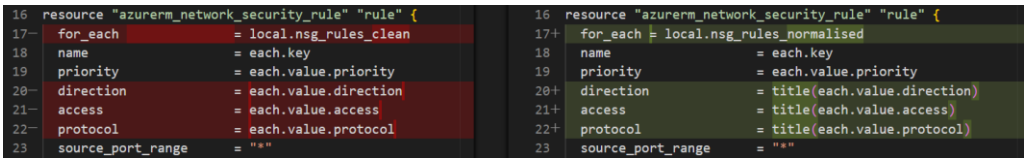
```
destination_ports_dropped = [
  for p in try(rule.destination_ports, []) : p
  if (
    length(trim(p, " -")) > 0 &&
    (
      length(regexall("[^0-9 -]", trim(p, " -"))) > 0 ||
      length(regexall("[0-9]+", trim(p, " -"))) == 0 ||
      length(regexall("[0-9]+", trim(p, " -"))) > 2
    )
  )
]
```

This block records any destination port entries that were ignored during sanitization. Non-empty inputs that contain invalid characters or an ambiguous number of port values are captured so they can be surfaced to the user as a warning during planning, rather than failing silently.

```
source_cidrs_dropped = [
  for c in try(rule.source_cidrs, []) : lower(trimspace(c))
  if (
    length(trimspace(c)) > 0 &&
    !can(cidrnetmask(trimspace(c)))
  )
]
```

Similarly, this bock records any source cidrs that were ignored during sanitization. These too are surfaced as output to the user during planning.

29. Compare the current **guided\main.tf** against the previous **phase3\main.tf**. The new main.tf is updated to use the new local.nsg_rules_normalised rather than local.nsg_rules_clean …




```
16  resource "azurerm_network_security_rule" "rule" {
17    for_each = local.nsg_rules_normalised
18    name                       = each.key
19    priority                   = each.value.priority
20    direction                  = title(each.value.direction)
21    access                     = title(each.value.access)
22    protocol                   = title(each.value.protocol)
23    source_port_range          = "*"
```

Noteworthy here is the use of 'title()' for direction, access and protocol. Azure requires these values begin with a capital letter; Inbound, Allow, Tcp etc., but our normalized version is all lowercased. We could have 'titled' the normalized version, but that violates the whole purpose of normalization.

Golden Rule: *Normalize input in locals; format it for the provider in the resource.*

30. Run planning …

    terraform plan -out=out4
    terraform show -json out4 | jq > out4.txt

31. Compare **out4.txt** against **out3.txt** and verify that naming and tags are now consistent and titled across all resources.

Normalizing field values ensure consistent provider interpretation regardless of user input formatting.

Phase 5 – Adding Guardrails

Now we will add variable validation and lifecycle conditions to block sanitized-but-unsafe inputs.

32. Close any open files.
33. Run **phases.cmd** and select 'Phase 5 - guardrails'

    Files variable.tf and main.tf are updated.

34. Review the four validations introduced in **variables.tf** ...

allowed_cidrs_http and **allowed_cidrs_https** both have the same validation of the proposed CIDRs..

```
10    validation {
11      condition = alltrue([
12        for c in var.allowed_cidrs_http :
13        can(cidrnetmask(trimspace(c)))
14      ])
15      error_message = "All HTTP CIDRs must be valid CIDR blocks."
16    }
```

The interesting portion here is **can(cidrnetmask(trimspace(c)))**

Terraform attempts to generate a subnet mask from the trimmed proposed CIDR. So /0 through /32 will succeed, all others will fail.

Variable **nsg_rules** has two validations...

```
31  variable "nsg_rules" {
32    type = map(object({ ···
40    }))
41
42    validation {
43      condition = alltrue([ ···
49      ])
50      error_message = "One or more NSG rules contain an invalid CIDR in source_cidrs."
51    }
52
53    validation {
54      condition = alltrue([ ···
57      ])
58      error_message = "All NSG rules must have a priority between 100 and 4096."
59    }
60  }
```

```
43    validation {
44      condition = alltrue([
45        for name, rule in var.nsg_rules :
46        alltrue([
47          for c in rule.source_cidrs :
48          can(cidrnetmask(trimspace(c)))
49        ])
50      ])
51      error_message = "One or more NSG rules contain an invalid CIDR in source_cidrs."
```

The first validation block checks that every value in source_cidrs is a valid CIDR (in the same way we validated the global allowed_cidrs_http and allowed_cidrs_https). However, in *this* lab it is effectively redundant because we already sanitise source_cidrs earlier and silently drop any invalid CIDRs using

can(cidrnetmask(...)). By the time validation runs, the invalid values are already gone.

This highlights a deliberate design choice:

- If the goal is **tolerant input handling**, keep sanitisation as-is (drop invalid CIDRs) and surface what was dropped via outputs.
- If the goal is **fail-fast enforcement**, sanitisation should only *normalise* (trim/lowercase), and should **not** filter using can(cidrnetmask). In that model, invalid CIDRs remain in the proposed list, and the validation block will fail loudly during terraform validate/plan, forcing the caller to correct the input.

In other words: you can either **accept-and-report** bad inputs, or **reject** them — but doing both at the same time is usually unnecessary. Terraform supports both tolerant and strict models, but you should choose one. Sanitizing *and* validating the same constraint is redundant and can obscure intent.

```
53    validation {
54      condition = alltrue([
55        for name, rule in var.nsg_rules :
56        rule.priority >= 100 && rule.priority <= 4096
57      ])
58      error_message = "All NSG rules must have a priority between 100 and 4096."
59    }
60  }
```

The second ensures the proposed priority is within valid ranges, here defined as being between 100 and 4096

35. Three lifecycle precondition checks are introduced against the NSG rules in **maint.tf**...

```
34    lifecycle {
35      # 0.0.0.0/0 prohibited
36    >   precondition {···
49      }
50
51      # ports must exist after sanitisation
52    >   precondition {···
55      }
56
57      # every numeric endpoint in each port spec must be 1-65535
58    >   precondition {···
67      }
68    }
```

36. Review these preconditions

```
35      # 0.0.0.0/0 prohibited
36      precondition {
37        condition = alltrue([
38          for c in concat(
39            each.value.source_cidrs,
40            each.value.allow_group == "http"
41            ? local.allowed_cidrs_http_clean
42            : each.value.allow_group == "https"
43            ? local.allowed_cidrs_https_clean
44            : []
45          ) : c != "0.0.0.0/0"
46        ])
47        error_message = "Rule '${each.key}' contains source CIDR (0.0.0.0/0)."
48      }
```

This precondition validates the final evaluated source CIDR set, not just the raw input field. It combines explicit source CIDRs with any additional CIDRs injected through the http or https allow-group logic, then asserts that no resulting entry is 0.0.0.0/0. This prevents accidental or indirect creation of an internet-open rule.

```
49      # ports must exist after sanitisation
50      precondition {
51        condition     = length(each.value.destination_ports) > 0
52        error_message = "Rule '${each.key}' has no destination ports after sanitisation."
53      }
54
55      # every numeric endpoint in each port spec must be 1-65535
```

This check validates that the destination_port_ranges attribute is non-empty after all prior transformation and sanitisation logic has been applied.

```
55      # every numeric endpoint in each port spec must be 1-65535
56      precondition {
57        condition = alltrue([
58          for p in each.value.destination_ports :
59          alltrue([
60            for n in regexall("[0-9]+", p) :
61            tonumber(n) >= 1 && tonumber(n) <= 65535
62          ])
63        ])
64        error_message = "Rule '${each.key}' contains destination ports outside the valid range (1-65535)."
65      }
```

This extracts all numeric values from each destination port definition (including ranges) and asserts that every value lies within 1–65535; rejects the rule if any value is out of bounds.

37. Run **terraform plan** using invalid values in terraform.tfvars to confirm Terraform now halts with clear error messages. Reset the values after testing.

    With all hygiene and guardrails in place, you can now use terraform console to explore your logic.

38. Run **terraform console** and test expressions such as:

    local.prefix
    local.allowed_cidrs_http_clean
    local.allowed_cidrs_https_clean
    keys(local.nsg_rules_clean)
    local.nsg_rules_normalised["allow-http-from-office"]

    Guardrails prevent risky misconfigurations from being applied — turning good hygiene into enforceable policy. Together, validation and preconditions demonstrate both defensive coding and policy enforcement.

Phase 6 – Let Us Get Real

In this lab we deliberately restricted the configuration to two global allow-lists — **allowed_cidrs_http** and **allowed_cidrs_https** and only one per rule. That was an artificial simplification designed to keep the focus on methodology, not quantity.

By working with a small, fixed number of lists, you concentrated on the key learning objectives:

- understanding how external data in terraform.tfvars can be messy or inconsistent
- cleaning and canonicalizing that input in locals
- normalising values before using them
- enforcing defensive checks through variable validation and resource preconditions

Everything we achieved with two groups - trimming spaces, enforcing lowercase, validating CIDRs, merging global and rule-level lists, and blocking unsafe values - remains completely valid when scaled up. The techniques you have applied are what matter, not the number of groups involved.

Moving Beyond Two Groups

In real environments, security teams rarely stop at just HTTP and HTTPS. They maintain many allow-lists — for example:

- web traffic from office networks
- administrative access from VPN ranges
- database access from application subnets
- “break-glass” emergency addresses

Terraform code should not change every time one of these groups is added; only the data should.

A More Realistic Exemplar

To illustrate how the same methodology scales, an exemplar is provided that uses a configurable **map of named allow-groups** rather than two fixed lists:

```
allow_groups = {
  http      = ["10.0.0.0/24", "10.0.1.0/24"]
  https     = ["10.0.2.0/24"]
  admin     = ["172.16.0.10/32"]
  breakglass = ["192.168.99.99/32"]
  ......
}
```

Each NSG rule specifies the groups it needs through an **allow_groups list**:

```
allow_groups = ["http", "https" ....]
```

Terraform then merges the CIDRs from all those groups along with any explicit per-rule sources:

```
source_address_prefixes = distinct(flatten(concat(
 each.value.source_cidrs,
 [for g in each.value.allow_groups : lookup(local.allow_groups_clean, g, [])]
)))
```

The underlying logic is unchanged - locals still canonicalize and clean data, validations still protect input, and preconditions still prevent unsafe deployment - but the model is now **flexible and extensible.** New groups can be added in terraform.tfvars without modifying Terraform code, and a single rule can include multiple groups.

39. Close any open files.
40. Run **phases.cmd** and select 'Phase 6 - exemplar'
41. Examine the exemplar code
42. Test it by manipulating terraform.tfvars and see how the code handles your input. Try to identify the huge validation deficiency that still exists (by design)

    This exemplar represents the tidy and normalised end-state of the same process you have just worked through; the same defensive hygiene, now applied to a structure that can grow with the environment. You began with an intentionally naïve design; you finish with a pattern ready for production.

Phase 7 – Challenge

Close the Priority Uniqueness Gap (NSG Rules)

Your NSG rules are now being defined as a map of rule objects, then normalised and applied via for_each. Most rule inputs are validated and/or checked before Terraform attempts to create resources. However, one important Azure constraint is not currently enforced by the module:

NSG rule priorities must be unique (within a direction).

This means you can accidentally define two inbound rules with the same priority and Terraform will happily plan the deployment — but Azure will reject it during apply.

Your task

43. Add a guardrail that prevents duplicate priorities from reaching Azure.
44. Your solution must:
    - Detect duplicate priorities before apply
    - Be correct for both Inbound and Outbound rules (even if this lab currently only uses Inbound)
    - Produce a clear error message when duplicates are found

How to test your solution

45. Introduce a deliberate duplication in terraform.tfvars by setting the same priority value on two rules with the same direction.
46. Run terraform plan.
47. Confirm that Terraform fails early with your message (rather than failing during apply with an Azure API error).

Option A: Variable validation (contract-level)

48. Implement the check inside **variables.tf** using a validation {} block in variable "nsg_rules".

    Why you might choose this ...

    - It fails earliest (input contract enforcement)
    - It is a clean “module interface” rule
    - It does not depend on derived values

    Trade-offs ...

    - Harder to include highly specific diagnostic details in the message

Option B: Resource precondition (boundary-level)

49. Implement the check inside **main.tf** using a lifecycle { precondition { ... } } block (or an additional precondition) on the NSG rule resource.

    Why you might choose this ...

- Keeps Azure-specific constraints close to the resource boundary
- Can use normalised locals (local.nsg_rules_normalised)
- Fits the pattern used for other “effective configuration” checks (ports, CIDR expansion)

Trade-offs …

- Evaluated later than variable validation
- Must be written carefully to avoid repeated logic or confusing errors

Completion criteria

50. You are finished when:

- Duplicate priorities in the same direction are caught during terraform plan
- Non-duplicates pass
- Your solution remains correct if outbound rules are later added

Solution

There is a solution folder under lab2/phases/exemplar. This contains a copy of main.tf and variables.tf with the priority check in place in each. Choose one of these, overwrite your current copy and uncomment the validation/precondition check as appropriate. Test by attempting to create two rules with the same priority. Comment out and then try the other method for comparison.

Phase 8 – Clean Up Any Mess Before You Leave!

51. If you deployed any resources: **terraform destroy -auto-approve**

Your work has not been in vain — this refined configuration forms the foundation for later labs focusing on effective use of modules.

Congratulations, you have completed the lab

Copyright